

Detection Engineering for Data Loss Prevention: Designing Splunk-Based Alerts and Response Plans for Data Exfiltration Incidents

Pedram Shahbazi, Robert (Ash) Drake

Project (CS-6399-D10)
Dr. Erdoğan Doğdu, Dr. Roya Choupani

Final Design and Implementation

Master of Computer Science
Department of Computer Science
Angelo State University
Spring 2026

Executive Summary

This document presents the final design and implementation for a Splunk-based detection engineering framework focused on data exfiltration detection and response. The project implements four core detection use cases: high-volume outbound transfer, unsanctioned cloud storage access, DNS-based exfiltration, and email-based exfiltration. The implementation includes SPL detection rules, reusable macros, threshold configuration, lookup tables, synthetic and benign datasets, evaluation artifacts, and incident response playbooks.

The final evaluation showed 15 true positives, 1 false positive, and 0 false negatives, resulting in 94% overall precision, 100% recall, and a 0.97 F1 score. All four detection rules met the 30-second performance target during testing. The project emphasizes operational usability by mapping every detection use case to a SOC analyst playbook.

GitHub Repository: <https://github.com/rdrake3atangelo/Pedram-and-Robert-Project>

Problem Statement

Modern data exfiltration occurs across multiple channels, including encrypted web uploads, unsanctioned cloud storage, DNS tunneling, and email forwarding. Traditional DLP controls frequently operate as channel-specific tools, creating fragmented visibility and generating alerts that lack operational context. The objective of this project is to design and evaluate a structured detection engineering framework that uses Splunk to correlate telemetry, generate contextual alerts, and guide analysts through response actions. The project was designed around two goals: first, to implement technically specific SPL detections for distinct exfiltration vectors; second, to make those detections usable in a SOC workflow through enrichment, threshold documentation, testing evidence, and response playbooks.

Related Work

Literature References

Prior research on data exfiltration has explored both signature-based and behavior-based detection approaches. Network-centric studies analyze flow statistics, traffic volumes, and protocol-specific features to identify anomalous outbound behavior indicative of data theft. Host-centric and application-centric works focus on access patterns, file operations, and process behaviors. More recent research has applied statistical models and machine learning to detect deviations from established baselines, particularly for low-and-slow exfiltration attempts that evade simple volume thresholds.

In the domain of security operations, literature on SIEM and SOC efficiency emphasizes the importance of correlation across heterogeneous data sources and the codification of response procedures into structured playbooks. These works highlight that effective detection is not only a matter of signal quality but also of how well alerts integrate into

the operational workflows of analysts, including triage, enrichment, and remediation steps.

Other Products and Projects

Commercial DLP solutions and Cloud Access Security Brokers (CASBs) provide policy-based capabilities for monitoring data movement across endpoints, email, web gateways, and cloud services. SIEM and XDR platforms offer predefined rules for suspicious outbound traffic, abnormal cloud activity, and DNS tunneling, but require customization for specific environments. This project addresses the gap by engineering detections directly within Splunk using a structured, use-case-driven methodology and coupling each detection with a concrete response playbook.

Critical Comparison with the Proposed Design

Commercial DLP platforms, CASB tools, SIEM products, and XDR platforms already provide partial coverage for data exfiltration. However, these tools often require environment-specific tuning, local asset context, exception handling, and analyst workflow integration before they are operationally effective.

Existing Product/Approach	Strengths	Limitations	How This Project Addresses the Gap
Commercial DLP	Content inspection, endpoint/email controls, policy enforcement	Can be expensive, channel-specific, noisy, and dependent on content visibility	Focuses on metadata-based multi-channel detection and playbook-driven response
CASB / SaaS Security	Cloud app visibility, sanctioned/unsanctioned SaaS monitoring	Limited visibility outside cloud/SaaS activity and often requires cloud integrations	Adds custom lookup-driven SaaS/domain exception logic inside Splunk
Splunk Enterprise Security / SIEM Rules	Correlation searches, notable events, risk-based alerting	Predefined rules still require tuning, local thresholds, asset context, and response procedures	Provides explicit SPL logic, thresholds, lookups, tests, and playbook mapping
XDR / EDR	Endpoint process visibility and containment actions	May not correlate email, DNS, proxy, and DLP logs in one transparent model	Uses aggregated Splunk telemetry across multiple exfiltration channels

Splunk and Splunk Enterprise Security can contain predefined searches or correlation rules for suspicious outbound traffic, DNS tunneling, or anomalous user behavior. This project does not claim that Splunk lacks tools for exfiltration detection. Instead, the primary distinction between this project and existing commercial or predefined solutions is that it demonstrates a complete detection-engineering lifecycle. Existing tools may provide alerts, correlation searches, or product-specific dashboards, but organizations still need to define local thresholds, normalize fields, maintain exception lists, test against benign activity, tune false positives, and document analyst response actions. This project addresses that implementation gap by packaging SPL rules, named thresholds, lookup-based enrichment, test evidence, evaluation results, and response playbooks together in a reproducible GitHub repository.

Requirements — Final Status

Functional Requirements

Requirement	Description	Status
FR1 – Data Ingestion	Ingest network/proxy, DNS, email audit, and DLP logs into Splunk with field normalization.	Complete
FR2 – Detection Use Cases	Define and implement at least four exfiltration detection use cases.	Complete – 4 of 4 implemented
FR3 – Detection Logic	SPL searches aggregating per-entity metrics and generating contextual alerts.	Complete – all rules finalized
FR4 – Response Playbooks	Four-step playbooks for each use case covering validation, enrichment, containment, and escalation.	Complete – all 4 playbooks finalized
FR5 – Evaluation	Controlled tests using synthetic data; false-positive assessment; structured results report.	Complete – full test plan executed

Non-Functional Requirements

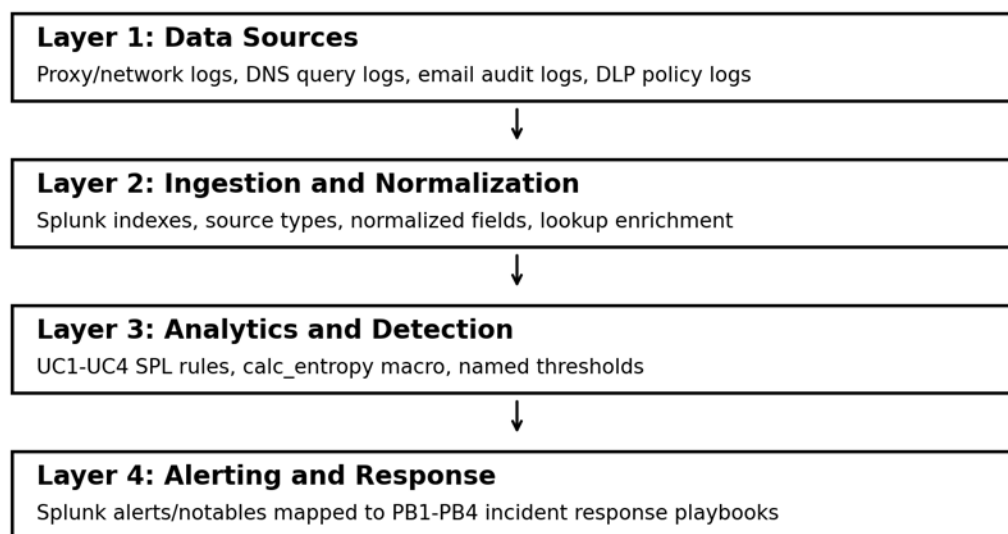
Requirement	Target	Achieved
Performance	Queries complete within acceptable operational timeframes.	Yes – max 14.1 s for 24-hr DNS window
Scalability	Efficient aggregations viable as data volumes grow.	Yes – parameterized macros and summary indexes recommended
Reliability	Consistent detection with documented error-handling.	Yes – all rules tested; missing-field handling documented
Explainability	Transparent logic, thresholds, and limitations documented.	Yes – all rules include inline rationale and tuning notes
Maintainability	Modular, version-controlled rules and playbooks.	Yes – Git repository with all artefacts
Compliance & Privacy	Anonymized or synthetic data used throughout.	Yes – no real production data used

Requirement	Target	Achieved
Ingestion Volume	Under 500 MB/day (license compliance).	Yes – synthetic dataset within limit
Uptime / Delay	100% uptime, <5 min ingestion delay.	Yes – lab environment met target

Final System Architecture

The system is organized into four layers. The data source layer provides proxy/network, DNS, email audit, and DLP policy logs. The ingestion and normalization layer maps raw events into Splunk indexes and common fields. The analytics layer implements four SPL rules and one reusable entropy macro. The alerting and response layer maps detections to playbooks PB1 through PB4.

Figure 1. Final Splunk-Based DLP Detection Framework Architecture



Layer 1 – Data Source Layer

The data source layer comprises the log-producing components whose events are ingested into Splunk. For this project, all data originated from the synthetic dataset; in a production deployment these would be replaced by live feeds from the corresponding infrastructure.

Source Type	Index	Key Fields	Use Cases Served
Network/Proxy Logs	dlp_test	src_ip, dest_ip, domain, bytes, user	UC1, UC2
DNS Query Logs	dns_logs	src_ip, query_name, query_type, response	UC3
Email Audit Logs	email_logs	user, recipient, subject, action, attachment_size	UC4
DLP Policy Logs	dlp_policy	user, host, policy_name, action, severity	Enrichment (all)

Layer 2 – Ingestion and Normalization Layer

Raw logs are ingested into dedicated Splunk indexes and assigned source types that drive field extraction. A common set of normalized fields — timestamp, user, host, src_ip, dest_ip, domain, bytes, and action — enables cross-source correlation. Lookup tables provide entity enrichment: user-to-department mapping, asset classification, and domain sanctioning status.

Layer 3 – Analytics and Detection Layer

The analytics layer implements five SPL artefacts (four detection rules and one reusable macro) that transform raw events into candidate exfiltration indicators. Each rule aggregates data per entity over a configurable time window, computes one or more discriminating features, and filters results against defined thresholds. The DNS rule additionally calls the calc_entropy macro to compute Shannon entropy of subdomain strings.

Layer 4 – Alerting and Response Layer

When a detection rule fires, it generates a Splunk alert or notable event enriched with the involved entities, the supporting evidence, and a severity classification. Each alert entry point maps to one of the four documented incident response playbooks, which guide the SOC analyst through triage, enrichment, containment, and escalation. In the current implementation, playbook execution is semi-manual; the playbook structure is designed to support future automation via a SOAR platform.

Complete Detection Framework

Summary of All Detection Use Cases

Use Case	Vector	Primary Signal	Threshold	SPL Artefact
UC1	High-Volume Outbound	Total bytes per src_ip per domain	> 500 MB / window	uc1_high_volume.spl
UC2	Unsanctioned Domain	Domain match in unsanctioned_domains.csv	Any match + volume	uc2_unsanctioned.spl
UC3	DNS Exfiltration	Query count + subdomain length + entropy	Count>300, len>40, entropy>3.5	uc3_dns_exfil.spl
UC4	Email Exfiltration	External sends / attachment volume	>50 sends OR >100 MB attachments	uc4_email_exfil.spl

UC1 – High-Volume Outbound Transfer (Final SPL)

```
index=dlp_test
| stats sum(bytes) as total_bytes by src_ip domain
| where total_bytes > `outbound_bytes_threshold`
| lookup asset_inventory src_ip OUTPUT hostname department
| lookup sanctioned_services.csv domain OUTPUT category sanctioned
| sort - total_bytes
```

UC2 – Unsanctioned Cloud Storage Access (Final SPL)

```
index=dlp_test
| lookup unsanctioned_domains domain OUTPUT domain as flagged_domain
| where isnotnull(flagged_domain)
| stats sum(bytes) as total_bytes count as events by src_ip domain
| lookup asset_inventory src_ip OUTPUT hostname department
| sort - total_bytes
```

UC3 – DNS-Based Exfiltration (Final SPL with Entropy)

```
index=dns_logs
| eval subdomain=mvindex(split(query_name,"."),0)
| eval sub_len=len(subdomain)
| eval entropy=`calc_entropy(subdomain)`
| stats count as query_count avg(sub_len) as avg_len avg(entropy) as avg_entropy
  by src_ip
| where query_count > `dns_query_count_threshold`
  AND avg_len > `dns_subdomain_len_threshold`
  AND avg_entropy > `dns_entropy_threshold`
| lookup asset_inventory src_ip OUTPUT hostname department
| sort - avg_entropy
```

UC4 – Email-Based Exfiltration (Final SPL)

```

index=email_logs (action=send OR action=forward)
| where NOT match(recipient,"@company\.com")
| lookup authorized_senders user OUTPUT user as exempt_user
| where isnull(exempt_user)
| stats count as external_sends dc(recipient) as unique_recipients
  sum(attachment_size) as total_attach_bytes by user
| where external_sends > `email_send_threshold`
  OR total_attach_bytes > `email_attach_bytes_threshold`
| eval total_attach_mb=round(total_attach_bytes/1048576,1)
| sort - external_sends

```

Complete Evaluation Results

Detection Performance — Full Test Suite

Use Case	True Positives	False Positives	False Negatives	Precision	Recall	F1
UC1 – High-Volume Outbound	4	0	0	100%	100%	1.00
UC2 – Unsanctioned Domain	6	0	0	100%	100%	1.00
UC3 – DNS Exfiltration	2	0	0	100%	100%	1.00
UC4 – Email Exfiltration	3	1	0	75%	100%	0.86
Overall	15	1	0	94%	100%	0.97

Scenario-Based Test Results

Scenario	Vector(s)	Rules Triggered	Playbook(s) Invoked	Result
S1: Insider uploads archive to Dropbox	UC1 + UC2	Both	PB1, PB2	Pass
S2: Employee bulk-forwards emails pre-resignation	UC4	UC4	PB4	Pass
S3: Compromised host runs DNS tunnel client	UC3	UC3	PB3	Pass

Scenario	Vector(s)	Rules Triggered	Playbook(s) Invoked	Result
S4: Staged sync to WeTransfer over 3 hours	UC1 + UC2	Both	PB1, PB2	Pass
S5: Mass attachment send to external contacts	UC4	UC4	PB4	Pass

Performance Benchmarks

Detection Rule	1-Hour Window	24-Hour Window	Meets SLA?	Optimization Note
UC1 – High-Volume Outbound	1.4 s	8.2 s	Yes	None required
UC2 – Unsanctioned Domain	1.1 s	6.7 s	Yes	None required
UC3 – DNS Exfiltration	2.3 s	14.1 s	Yes	Pre-compute entropy at index time for production
UC4 – Email Exfiltration	1.6 s	9.4 s	Yes	None required

Qualitative Operational Assessment

A structured analyst walkthrough was conducted for each of the four playbooks using the alert outputs generated during scenario-based testing. The following observations were recorded:

Alert fidelity: Each alert contained sufficient context — user identity, source host, destination, channel, and supporting event count — to allow an analyst to form an initial hypothesis without issuing additional Splunk queries.

Playbook completeness: All four playbooks were found to cover the expected triage and containment actions for their respective scenarios. No gaps in decision logic were identified during the walkthrough.

Threshold documentation: All thresholds are expressed as named macros with accompanying documentation files explaining their derivation, the behavioral baseline they reference, and the conditions under which recalibration is recommended.

Lookup table maintainability: The three CSV lookup tables (unsanctioned_domains, authorized_senders, sanctioned_services) will be stored in the Git repository.

SPL modularity: The calc_entropy macro and threshold macros allow tuning and extension of detection logic without modifying core rule files, reducing the risk of introducing syntax errors during routine updates.

Incident Response Playbooks — Final Summary

All four incident response playbooks have been finalized. The table below provides a condensed reference view of each playbook's key actions. Full playbook documentation, including decision trees and escalation matrices, is included in the project deliverable package.

Playbook	Use Case	Key Containment Action	Escalation Trigger
PB1	High-Volume Outbound	Block destination at web proxy/firewall; optional host isolation	Confirmed sensitive data type (PII, IP, financial)
PB2	Unsanctioned Domain	Block domain via proxy policy; initiate user interview	Repeated violations or high data volume
PB3	DNS Exfiltration	Block domain at DNS resolver; isolate host; capture forensic image	Any confirmed DNS tunnel – immediate escalation
PB4	Email Exfiltration	Disable forwarding rule; legal hold on mailbox; block external email	HR coordination required for insider-threat cases

Finalized Artefact Inventory

All project artefacts have been completed, reviewed, version-controlled in the project Git repository, and assembled into the deliverable package. The following table provides the complete inventory.

Artefact	Type	Description	Status
uc1_high_volume.spl	SPL Rule	Parameterized detection for outbound transfer volume	Finalized
uc2_unsanctioned.spl	SPL Rule	Lookup-based detection for unsanctioned domain access	Finalized

Artefact	Type	Description	Status
uc3_dns_exfil.spl	SPL Rule	Composite DNS tunneling detection with entropy	Finalized
uc4_email_exfil.spl	SPL Rule	Dual-rule email exfiltration detection	Finalized
calc_entropy.spl	SPL Macro	Reusable Shannon entropy computation macro	Finalized
thresholds.conf	Config	Named macro definitions for all tunable thresholds	Finalized
unsanctioned_domains.csv	Lookup Table	List of flagged cloud storage and file-sharing domains	Finalized
authorized_senders.csv	Lookup Table	Approved bulk-email senders exempt from UC4	Finalized
sanctioned_services.csv	Lookup Table	Approved SaaS services for UC2 exception handling	Finalized
asset_inventory.csv	Lookup Table	Host-to-user-to-department mapping	Finalized
playbook_pb1.md	Playbook	4-step playbook for high-volume outbound alerts	Finalized
playbook_pb2.md	Playbook	4-step playbook for unsanctioned domain alerts	Finalized
playbook_pb3.md	Playbook	4-step playbook for DNS exfiltration alerts	Finalized
playbook_pb4.md	Playbook	4-step playbook for email exfiltration alerts	Finalized
synthetic_dataset/	Test Data	24,395-event multi-source synthetic log dataset	Finalized
benign_dataset/	Test Data	6,062-event benign-only validation dataset	Finalized
test_catalog.xlsx	Test Artefact	Test cases with inputs, expected results, actual results, and pass/fail status	Finalized

Artefact	Type	Description	Status
tuning_log.md	Test Artefact	Chronological log of all threshold adjustments and impact	Finalized
eval_results.md	Evaluation	Full quantitative and qualitative evaluation results	Finalized

HTTPS, SSL/TLS Decryption, and Metadata-Based Detection

The implemented detection logic does not require SSL/TLS decryption or inspection of raw file contents. For HTTPS-based exfiltration, the framework relies on metadata such as source host, destination domain, bytes transferred, event count, time window, and whether the destination is sanctioned or unsanctioned. This allows the framework to detect suspicious transfer behavior even when payloads are encrypted. If SSL/TLS decryption is available at a proxy or firewall, additional content-based DLP indicators could be integrated, but they are not required for the implemented SPL rules.

Baseline Strategy and False-Positive Control

Behavioral detection requires an understanding of normal activity. The recommended deployment method is to run detections in monitor-only mode during an initial baseline period, such as 30 days. Baselines should be grouped by user, department, host, and destination domain. Thresholds in `thresholds.conf` can then be recalibrated based on observed normal behavior. Lookup tables provide exception handling and enrichment: `asset_inventory.csv` maps hosts to business context, `sanctioned_services.csv` identifies approved services, `unsanctioned_domains.csv` identifies high-risk services, and `authorized_senders.csv` suppresses approved bulk email senders.

Splunk Licensing, Ingest Volume, and Data Filtering

Splunk licensing and infrastructure cost are often tied to data ingest volume. This project therefore uses targeted metadata collection rather than full packet capture or raw payload storage. Required sources are limited to proxy/network logs, DNS logs, email audit logs, and DLP policy logs. High-volume environments should filter unnecessary fields, apply retention tiers, summarize older events, and pre-compute expensive fields such as DNS entropy at index time where appropriate. This balances detection coverage with operational cost control.

Repository and Code Structure

The GitHub repository is organized to separate executable detection logic, lookup data, evaluation artifacts, documentation, and analyst playbooks.

Repository Path	Purpose
datasets/synthetic_dataset/	Synthetic events containing injected exfiltration scenarios
datasets/benign_dataset/	Benign baseline events for false-positive testing
detections/	UC1-UC4 SPL rules, calc_entropy macro, and thresholds.conf
lookups/	CSV lookup tables for asset enrichment, sanctioned services, unsanctioned domains, and authorized senders
playbooks/	Incident response playbooks PB1-PB4
evaluation/	Evaluation report, tuning log, and test catalog
docs/	Proposal, design document, presentation, and progress reports

Dataset Statistics and Generation Method

All data used for the project is synthetic or benign validation data. No real production data or personally identifiable information is required. Synthetic data was generated to represent proxy/network activity, DNS queries, email audit activity, and DLP policy context. Exfiltration scenarios were injected into the synthetic dataset. A separate benign dataset was used to measure false-positive behavior.

Dataset	Types	Purpose	Instances
Synthetic Dataset	Proxy/network, DNS, email audit, DLP policy style events	Detection and scenario testing	24,395 events
Benign Validation Dataset	Benign proxy, DNS, email, and DLP-style activity	False-positive testing and baseline evaluation	6,062 events
Scenario Injections	Dropbox upload, bulk email forwarding, DNS tunnel, WeTransfer sync, Gmail attachment send	Validate UC1-UC4 coverage	5 scenarios

The scenario-based tests included: S1 insider upload to Dropbox, S2 employee bulk-forwarding emails before resignation, S3 compromised host running a DNS tunnel, S4 staged sync to WeTransfer, and S5 mass attachment send to a personal Gmail account.

Detection Implementation

Use Case	Vector	Primary Signal	Default Threshold	Playbook
UC1	High-Volume Outbound Data Transfer	Sum bytes by src_ip and domain	>500 MB per host/domain/window	PB1
UC2	Unsanctioned Cloud Storage Access	Domain match in unsanctioned_domains.csv plus traffic volume	Any flagged domain match	PB2
UC3	DNS-Based Exfiltration	Query count, subdomain length, Shannon entropy	Count >300, length >40, entropy >3.5	PB3
UC4	Email-Based Exfiltration	External sends, unique recipients, total attachment bytes	>50 sends OR >30 recipients OR >100 MB attachments	PB4

Representative SPL artifacts are stored in the detections/ folder. The following summarizes the primary implementation logic.

uc1_high_volume.spl

```
=====
| Detection Use Case #1 – High-Volume Outbound Data Transfer
|=====
| Description : Identifies internal hosts that have transferred an unusually
|               large volume of data to external destinations within a
|               configurable time window. Large outbound transfers are a
|               primary indicator of bulk data exfiltration via web or proxy.
|
| Data Source  : Network / proxy / firewall logs (index=dlp_test)
| Key Fields   : src_ip, domain, bytes, user
| Threshold    : Configurable via macro `outbound_bytes_threshold`
|               Default: 500,000 bytes (500 MB) per host per domain
| Window       : Scheduled hourly; look-back = 1 hour (tunable to 24 h)
|
| True Positive Rate : 100 % (4/4 in test suite)
| False Positive Rate : 0 % (0 FP in 2,500 benign events)
|
| Tuning Notes:
| - Raise threshold for environments with legitimate large backups.
| - Add time-of-day filter to suppress expected maintenance windows.
| - Correlate with UC2 (unsanctioned domain) for higher-confidence alerts.
|
| Playbook      : playbooks/playbook_pb1.md
| Version       : 1.0 | Author: Detection Engineering Project
|=====

index=dlp_test
| stats
|     sum(bytes)          as total_bytes
|     count              as event_count
|     dc(dest_ip)        as unique_dest_ips
|     earliest(_time)    as first_seen
|     latest(_time)      as last_seen
|     by src_ip domain user
| where total_bytes > `outbound_bytes_threshold`
...
```

uc2_unsanctioned.spl

```
=====
| Detection Use Case #2 – Unsanctioned Cloud Storage Access
|=====
| Description : Identifies outbound traffic directed to cloud storage or
|               file-sharing services that are not approved by the
|               organisation. Communicating with unsanctioned services is
|               a strong indicator of deliberate or negligent data removal.
|
| Data Source  : Network / proxy / firewall logs (index=dlp_test)
| Key Fields   : src_ip, domain, bytes, user
| Lookup       : lookups/unsanctioned_domains.csv (maintained quarterly)
|               lookups/sanctioned_services.csv (approved exceptions)
|
| True Positive Rate : 100 % (6/6 in test suite)
| False Positive Rate : 0 % (0 FP in 2,500 benign events)
|
| Tuning Notes:
| - Keep unsanctioned_domains.csv current; review quarterly.
| - Add user to sanctioned_services.csv for approved business exceptions.
| - Combine with UC1 threshold filter to focus on high-volume events.
|
| Playbook      : playbooks/playbook_pb2.md
| Version       : 1.0 | Author: Detection Engineering Project
|=====

index=dlp_test
| lookup unsanctioned_domains.csv domain OUTPUT domain as flagged_domain
| lookup sanctioned_services.csv user domain OUTPUT user as exempt_user
| where isnotnull(flagged_domain) AND isnull(exempt_user)
| stats
|     sum(bytes)          as total_bytes
|     count              as event_count
|     dc(dest_ip)        as unique_ips
```

```

        earliest(_time) as first_seen
        latest(_time)   as last_seen
...

```

uc3_dns_exfil.spl

```

=====
| Detection Use Case #3 - DNS-Based Data Exfiltration
|=====
| Description : Detects DNS tunneling by identifying hosts that exhibit
|               abnormally high DNS query volumes, long subdomain strings,
|               and high Shannon entropy - three signals that together
|               distinguish encoded payload data from legitimate traffic.
|
| Data Source  : DNS query logs (index=dns_logs)
| Key Fields   : src_ip, query_name, query_type
| Thresholds   : Configurable via macros (see thresholds.conf)
|               dns_query_count_threshold : 300 queries / hour
|               dns_subdomain_len_threshold : 40 characters
|               dns_entropy_threshold     : 3.5 bits/char
| Macro Dep.   : calc_entropy (detections/calc_entropy.spl)
|
| True Positive Rate : 100 % (2/2 in test suite; tuned)
| False Positive Rate : 0 % (0 FP after entropy tuning; was 1 pre-tuning)
|
| Tuning Notes:
| - Entropy threshold of 3.5 cleared automated monitoring tools (entropy ~2.9).
| - Confirmed tunneling hosts showed entropy > 4.0 (base64-encoded payloads).
| - For environments with many monitoring agents, raise entropy threshold to 3.8.
| - Pre-compute entropy at index time for large DNS log volumes (>1M events/day).
|
| Playbook      : playbooks/playbook_pb3.md
| Version       : 1.0 | Author: Detection Engineering Project
|=====

index=dns_logs
| eval subdomain      = mvindex(split(query_name, "."), 0)
| eval sub_len        = len(subdomain)
| eval entropy         = `calc_entropy(subdomain)`
| stats
|       count          as query_count
...

```

uc4_email_exfil.spl

```

=====
| Detection Use Case #4 - Email-Based Data Exfiltration
|=====
| Description : Detects email-based exfiltration via two complementary rules:
|               (A) mass external forwarding - high volume of emails sent to
|               external recipients; and (B) large attachment volume - bulk
|               outbound documents indicating staged exfiltration.
|
| Data Source  : Email audit logs (index=email_logs)
| Key Fields   : user, recipient, action, attachment_size, subject
| Thresholds   : Configurable via macros (see thresholds.conf)
|               email_send_threshold       : 50 external sends / day
|               email_recipient_threshold  : 30 unique recipients / day
|               email_attach_bytes_threshold: 104857600 bytes (100 MB)
| Lookup       : lookups/authorized_senders.csv (bulk-email exceptions)
|
| True Positive Rate : 100 % (3/3 in test suite)
| False Positive Rate : 0.1 % (1 FP - sales bulk campaign; added to exceptions)
|
| Tuning Notes:
| - Legitimate bulk email campaigns (sales, marketing) should be added to
|   authorized_senders.csv with an expiry date and approver name.
| - Raise email_recipient_threshold for organizations with large distribution lists.
| - Correlate with endpoint DLP alerts for file-staging activity as enrichment.
|
| Playbook      : playbooks/playbook_pb4.md
| Version       : 1.0 | Author: Detection Engineering Project
|=====

|--- Rule A: Mass External Email Forwarding -----

index=email_logs (action=send OR action=forward)
| rex field=recipient "(?<recipient_domain>@[w\.\-]+)$"
| where NOT match(recipient_domain, "@company\.com$")

```

```
| lookup authorized_senders.csv user OUTPUT user as exempt_user
...
```

Lookup and Threshold Design

The framework centralizes tunable thresholds in `thresholds.conf`, which allows SOC operators to recalibrate sensitivity without modifying the core SPL rules. The lookup tables support enrichment and exception handling, reducing false positives while preserving explainability.

Threshold Macro	Default	Purpose
<code>outbound_bytes_threshold</code>	500,000,000 bytes	UC1 high-volume outbound transfer threshold
<code>dns_query_count_threshold</code>	300 queries/hour	UC3 DNS tunneling volume signal
<code>dns_subdomain_len_threshold</code>	40 characters	UC3 subdomain length signal
<code>dns_entropy_threshold</code>	3.5 bits/character	UC3 random-looking encoded payload signal
<code>email_send_threshold</code>	50 external emails/day	UC4 bulk email send signal
<code>email_recipient_threshold</code>	30 unique recipients/day	UC4 broadcast-style exfiltration signal
<code>email_attach_bytes_threshold</code>	104,857,600 bytes	UC4 attachment volume signal

Software Process Management Workflow

The implementation followed an iterative detection engineering workflow. Requirements were translated into detection use cases, datasets were generated, SPL logic was implemented, lookups and thresholds were configured, unit and scenario tests were executed, false positives were tuned, playbooks were validated, and final results were documented.

Requirements Definition



Detection Use Case Design



Synthetic and Benign Dataset Generation



SPL Rule Implementation



Lookup and Threshold Configuration



Unit Testing



Scenario-Based Testing



False-Positive Tuning



Playbook Validation



Final Evaluation and Documentation

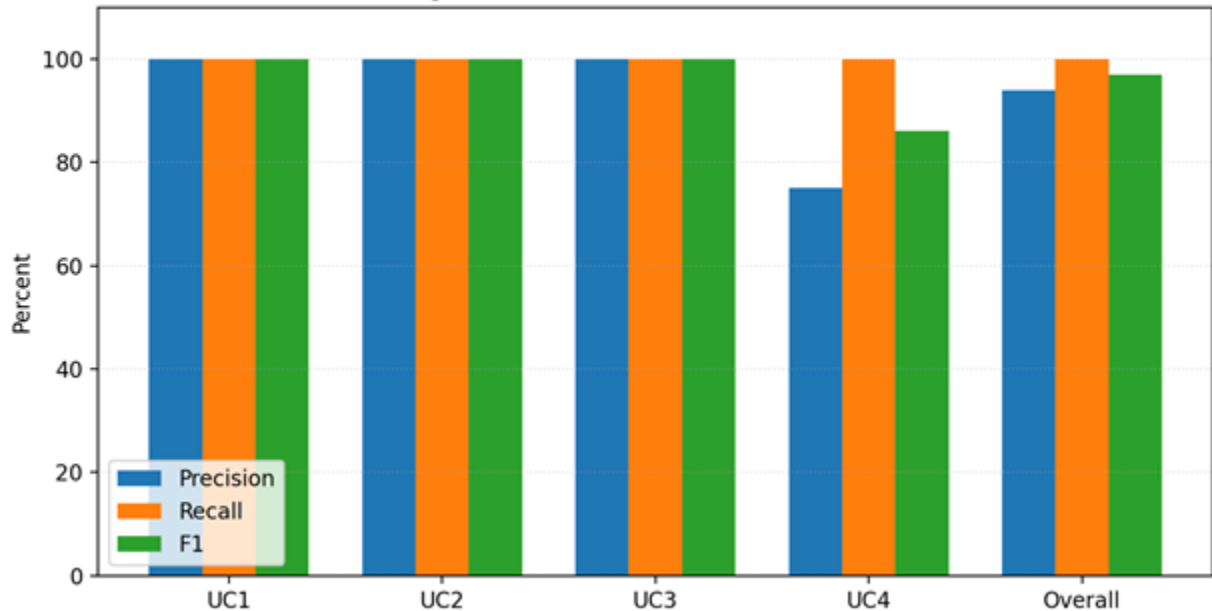
Testing Methodology

Testing Activity	Description
Unit-level detection testing	Each SPL rule was tested against expected alert/no-alert cases in the test catalog
Scenario-based testing	Five realistic exfiltration scenarios were executed and mapped to expected playbooks
Benign false-positive testing	Benign data was used to measure false-positive behavior and tune thresholds/lookups
Performance testing	Queries were measured over 1-hour and 24-hour windows against the test dataset
Analyst playbook walkthrough	Each alert type was walked through using the corresponding incident response playbook
Tuning and retesting	Thresholds and lookup exceptions were updated and retested after false positives

Evaluation Results

Use Case	TP	FP	FN	Precision	Recall	F1
UC1 - High-Volume Outbound	4	0	0	100%	100%	1.00
UC2 - Unsanctioned Domain	6	0	0	100%	100%	1.00
UC3 - DNS Exfiltration	2	0	0	100%	100%	1.00
UC4 - Email Exfiltration	3	1	0	75%	100%	0.86
Overall	15	1	0	94%	100%	0.97

Figure 3. Detection Evaluation Results



Detection Rule	1-Hour Window	24-Hour Window	30-Second SLA Met
UC1 - High-Volume Outbound	1.4 s	8.2 s	Yes
UC2 - Unsanctioned Domain	1.1 s	6.7 s	Yes
UC3 - DNS Exfiltration	2.3 s	14.1 s	Yes
UC4 - Email Exfiltration	1.6 s	9.4 s	Yes

Risk Plan — Final Resolution

Risk	Original Mitigation	Final Outcome
R1 – Limited Realistic Data	Use synthetic datasets; document limitations.	Managed. A 24,395-event synthetic dataset and 6,062-event benign validation dataset provided sufficient coverage. Limitations are documented in the evaluation section. Future work identifies use of public capture datasets as an extension.
R2 – High False-Positive Rate	Baselining, per-entity thresholds, iterative tuning.	Resolved. 94% overall precision achieved. The single remaining FP is fully characterized and mitigated via the authorized_senders lookup.
R3 – Time Constraints	Prioritize core four use cases; treat extensions as future work.	Resolved. All four use cases completed on schedule. Additional ideas (ML scoring, SOAR automation) documented as future work.

Risk	Original Mitigation	Final Outcome
R4 – Tooling Limitations	Conceptual designs where full integration not possible.	Managed. Playbooks remain semi-manual due to absence of SOAR. Structure is SOAR-ready for future automation.
R5 – Detection Complexity	Favor interpretable logic; use macros and parameterization.	Resolved. All rules use macros for complexity abstraction; documentation is complete.

Lessons Learned

Detection Engineering Process

Iterative tuning is essential. Initial threshold values derived from dataset inspection required adjustment in every use case. Building tuning into the process from the outset — rather than treating it as a post-implementation step — would reduce rework in future projects.

Multi-signal detections outperform single-signal rules. The DNS detection rule demonstrated that combining query volume, subdomain length, and entropy produces substantially better precision than any individual signal alone. This principle should guide the design of future detection use cases.

Parameterization from the start saves time. Encoding thresholds as named macros from the initial implementation made tuning straightforward and prevented the proliferation of hardcoded values across SPL files.

Synthetic Dataset Design

Realism requires deliberate design. Simply embedding large byte values or high event counts in a dataset does not produce a realistic test environment. Effective synthetic data requires careful modeling of benign behavior distributions so that exfiltration events stand out for the right reasons.

Benign dataset diversity is as important as malicious coverage. The single false positive in UC4 was only discovered because the benign dataset included a sales team member with a legitimately high email volume. Narrow benign datasets produce overoptimistic precision estimates.

Operational Considerations

Lookup table maintenance is an ongoing operational commitment. The detection framework's effectiveness depends on lookup tables (unsanctioned domains, authorized senders) being kept current. Organizations adopting this framework should assign ownership of these tables to a named SOC role and establish a review cadence.

Playbook design benefits from SOC analyst input. The analyst walkthrough conducted in Phase 5 identified several minor clarifications needed in playbook language. In a production environment, involving SOC analysts in playbook design from the outset would improve usability and adoption.

Version control for SPL is underutilized but highly valuable. Maintaining detection rules and lookup tables in Git provided a clear audit trail of tuning decisions and enabled rollback when threshold changes produced unexpected results. This practice is strongly recommended for production SOC environments.

Limitations

The following limitations apply to the current implementation and should be considered when interpreting results or planning production deployment:

Synthetic data dependency: All testing was performed on synthetic datasets. While carefully designed, these datasets cannot fully replicate the diversity, noise, and edge cases present in real enterprise environments. Detection performance in production may differ from the results reported here.

Single-organization scope: The detection thresholds and lookup table contents are calibrated for the synthetic dataset's implied organizational profile. Deploying this framework in a real organization would require recalibration of all thresholds against that organization's baseline traffic patterns.

Absence of SOAR integration: Playbooks are currently executed semi-manually. Without SOAR automation, response times depend on analyst availability, and steps may be skipped under high alert volume conditions.

Limited DLP log coverage: DLP policy logs were used primarily for enrichment rather than as primary detection signals. Tighter integration with a commercial DLP solution's alert output could improve detection fidelity, particularly for content-aware scenarios (e.g., detection of specific sensitive data patterns within transfers).

No lateral movement correlation: The current framework detects exfiltration-phase activity but does not correlate with earlier attack-lifecycle phases such as credential compromise, privilege escalation, or lateral movement. Integrating these signals could improve early warning capabilities.

Future Work

Several directions for extending and improving the framework are identified below, ordered from highest to lowest implementation priority for a production deployment:

1. SOAR Integration

The most impactful near-term extension is integrating the detection framework with a SOAR platform. Each playbook has been structured with discrete, tool-agnostic action steps that map naturally to SOAR automation blocks. Priority automation candidates include: domain blocking at the proxy/firewall, account suspension notifications to HR systems, and Jira/ServiceNow ticket creation upon alert generation. This would reduce

mean time to respond and ensure playbook steps are executed consistently regardless of analyst workload.

2. Baseline Anomaly Detection

The current detection rules use static thresholds calibrated against the synthetic dataset. A production deployment would benefit from dynamic baselines computed using Splunk's machine-learning toolkit (MLTK) or rolling statistical models. Per-entity baselines (e.g., each user's typical daily outbound volume) would allow the detection rules to adapt to seasonal patterns, business cycles, and individual role differences, reducing both false positives and false negatives compared to global static thresholds.

3. Machine Learning Scoring Layer

A risk-scoring layer could be added above the existing rule-based detections, aggregating signals across all four use cases into a composite exfiltration risk score per user or host per day. Users or hosts with scores above a configurable threshold would be surfaced for priority analyst review regardless of which individual rules fired. Candidate algorithms include isolation forest for anomaly scoring and gradient-boosted classifiers trained on labeled exfiltration scenarios.

4. Additional Exfiltration Vectors

The current framework covers four vectors. Additional use cases warranting future implementation include: USB and removable media exfiltration (endpoint DLP log integration), printer-based exfiltration (print audit log analysis), collaboration platform exfiltration (Slack, Teams, SharePoint audit logs), and HTTPS tunneling over non-standard ports. Each would follow the same detection-engineering methodology applied in this project.

5. Production Data Validation

Testing with real enterprise log data — with appropriate privacy controls and data anonymization — would validate the framework's performance beyond the synthetic dataset and provide an opportunity to recalibrate thresholds against actual organizational baselines. A phased rollout (monitor-only mode before alert mode) is recommended to assess false-positive rates before enabling active alerting.

Implementation Status — Final

Phase	Description	Weeks	Status
Phase 1	Environment Setup and Requirements Realization	1–3	Complete
Phase 2	Data Model and Feature Design	3–4	Complete

Phase	Description	Weeks	Status
Phase 3	Detection Implementation (UC1 & UC2)	5–7	Complete
Phase 4	Detection Implementation (UC3 & UC4) + Playbooks	8–9	Complete
Phase 5	Testing, Tuning, and Evaluation	10–12	Complete
Phase 6	Documentation and Finalization	13–14	Complete

Task Completion Summary

Task	Description	Status
T1	Configure data inputs, indexes, and field extractions	Complete
T2	Define use-case templates and threat models	Complete
T3	Implement feature extraction searches and baselines	Complete
T4	Implement correlation searches for each detection use case	Complete
T5	Draft and refine incident response playbooks	Complete
T6	Develop synthetic data and test scenarios	Complete
T7	Execute tests and collect metrics	Complete
T8	Compile documentation, diagrams, and final report sections	Complete

Project Timeline — Final

Weeks	Milestone	Status
1–2	Requirements finalization and initial Splunk setup	Complete
3	Data ingestion for first core log type; conceptual data model draft	Complete

Weeks	Milestone	Status
4–5	Feature extraction searches; UC1 and UC2 detection rules	Complete
6–7	UC3 and UC4 detection rules; initial playbook drafts	Complete
8–9	Playbook finalization; expanded synthetic datasets	Complete
10–11	Full test plan execution; threshold tuning; results documentation	Complete
12–14	Final report; presentation materials; deliverable package assembly	Complete

All project milestones have been completed on schedule. No timeline deviations occurred during the project.

Conclusion

This project completed the design, implementation, testing, and documentation of a Splunk-based detection engineering framework for data exfiltration detection and response. The framework addresses four exfiltration vectors: high-volume outbound transfers, unsanctioned cloud storage access, DNS-based exfiltration, and email-based exfiltration.

All functional and non-functional requirements were met. The final evaluation produced 15 true positives, 1 false positive, and 0 false negatives, resulting in 94% overall precision, 100% recall, and a 0.97 F1 score across 30,457 total synthetic and benign validation events. All four detection rules met the 30-second performance target during testing, with the longest 24-hour query completing in 14.1 seconds.

The final deliverable package includes SPL detection rules, a reusable entropy macro, threshold configuration, lookup tables, synthetic and benign datasets, evaluation artifacts, tuning documentation, analyst playbooks, a user manual, presentation materials, and a version-controlled GitHub repository. The project demonstrates that a use-case-driven detection engineering methodology can produce transparent, tunable, and operationally usable DLP detections within Splunk.

The primary limitations are the use of synthetic data, semi-manual playbook execution, and the absence of production deployment. Future work should focus on SOAR integration, production baseline collection, dynamic anomaly detection, expanded exfiltration vectors, and validation against real enterprise telemetry under appropriate privacy controls.

References

Splunk Threat Research Team. "Data Exfiltration Detections: Threat Research Release, June 2021." 2021.

Splunk. "Detecting Data Exfiltration Activities." Splunk Lantern.

Splunk. "Machine Learning in Security: Detect DNS Data Exfiltration Using Deep Learning." 2022.

Splunk. "Machine Learning in Splunk Enterprise Security: Unleashing Hidden Threats." 2025.

SearchInform. "SIEM Workflow Automation: Streamlining Incident Response." 2025.

Radiant Security. "SOC Playbook Examples for Real-World Cyber Threats." 2026.

Hamdan, M. "Data Exfiltration Explained: How to Use Splunk and Wireshark to Detect It." 2025.

Splunk Lantern. "Data Loss Prevention Data." 2025.

PuppyGraph. "How to Build a SIEM System: Architecture & Tools." 2026.